



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Arboles en C++
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero – “B”
Alumnos participantes:	Moyolema Criollo Katherine Lissette
Docente:	Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivo

Implementar en C++ un Árbol Binario de Búsqueda — BST para organizar empleados de una empresa usando un código numérico como clave. El ejercicio se relaciona con la diapositiva del organigrama empresarial, donde se identifican raíz, niveles, nodos internos y hojas.

2.2 Problema práctico

Una empresa necesita registrar empleados en una estructura jerárquica usando un árbol binario de búsqueda.

Cada empleado tendrá: código, nombre, cargo

El árbol debe permitir:

- Insertar empleados.
- Buscar un empleado por código.
- Mostrar recorrido inorden.
- Mostrar recorrido preorden.
- Mostrar recorrido postorden.
- Calcular altura del árbol.
- Identificar hojas.
- Mostrar la raíz del árbol.

2.3 Estructura del proyecto

Tu proyecto tiene esta estructura:

- **Carpeta src/:** Contiene todos los archivos de código fuente
 - **ArbolBST.cpp:** Implementación de todos los métodos del árbol
 - **ArbolBST.h:** Cabecera de la clase ArbolBST
 - **Empleado.h:** Definición del struct Empleado
 - **Nodo.cpp:** Implementación del constructor del Nodo
 - **Nodo.h:** Definición del struct Nodo
 - **main.cpp:** Menú principal y lógica de interacción con el usuario
- **Carpeta Capturas/:** Contiene las 9 capturas de pantalla de la ejecución
- **README.md:** Documentación completa del proyecto

2.4 Implementación técnica

2.4.1 Estructuras y clases utilizadas

Struct Empleado:

- **codigo (int):** Identificador único del empleado
- **nombre (string):** Nombre del empleado
- **cargo (string):** Cargo dentro de la empresa



Struct Nodo:

- **dato (Empleado):** Información del empleado
- **izquierdo (Nodo*):** Puntero al hijo izquierdo
- **derecho (Nodo*):** Puntero al hijo derecho
- **Constructor:** Inicializa el nodo con un empleado y pone los punteros a nullptr

Clase ArbolBST:

- **raiz (Nodo*):** Puntero al nodo raíz del árbol
- **Métodos privados:** insertar, buscar, inorden, preorden, postorden, altura, mostrarHojas
- **Métodos públicos:** insertarEmpleado, buscarEmpleado, mostrarRaiz, mostrarInorden, mostrarPreorden, mostrarPostorden, mostrarAltura, mostrarNodosHoja

2.4.2 Métodos implementados

- **insertar:** Agrega un nuevo empleado al árbol respetando la regla BST (menores a la izquierda, mayores a la derecha). Si el código ya existe, muestra un mensaje de error.
- **buscar:** Recorre el árbol comparando el código buscado con los nodos. Retorna el nodo si lo encuentra o nullptr si no existe.
- **inorden:** Recorre el árbol en orden: izquierdo - raíz - derecho. Muestra los empleados ordenados por código de menor a mayor.
- **preorden:** Recorre el árbol en orden: raíz - izquierdo - derecho. Útil para copiar el árbol.
- **postorden:** Recorre el árbol en orden: izquierdo - derecho - raíz. Útil para eliminar el árbol.
- **altura:** Calcula recursivamente la altura del árbol. Retorna 0 si el árbol está vacío, o $1 + \max(\text{alturaIzq}, \text{alturaDer})$.
- **mostrarHojas:** Recorre el árbol e imprime solo los nodos que no tienen hijos izquierdo ni derecho.

2.5 Funcionalidades

Funcionalidad	Estado
Insertar empleados	Sí
Buscar empleados por código	Sí
Mostrar la raíz del árbol	Sí
Recorrido inorden	Sí
Recorrido preorden	Sí
Recorrido postorden	Sí
Calcular altura del árbol	Sí
Mostrar nodos hoja	Sí

2.6 Datos de prueba

Código	Nombre	Cargo
50	Empresa UTA	Raíz
30	Gerente Ventas	Nodo interno
70	Gerentes Finanzas	Nodo interno
20	Emp1	Hoja
40	Emp2	Hoja
60	Emp3	Hoja
80	Emp4	Hoja



Resultados esperados:

- **Inorden:** 20, 30, 40, 50, 60, 70, 80
- **Preorden:** 50, 30, 20, 40, 70, 60, 80
- **Postorden:** 20, 40, 30, 60, 80, 70, 50
- **Altura:** 3
- **Hojas:** 20, 40, 60, 80

2.7 Capturas de ejecución

- **Captura 1:** Menú principal del programa

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
```

- **Captura 2:** Inserción de un nuevo empleado

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
Codigo: 50
Nombre: Empresa UTA
Cargo: Raiz

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
Codigo: 30
Nombre: Gerente Ventas
Cargo: Nodo Interno
```

- **Captura 3:** Búsqueda de empleado por código

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 2
Ingresa codigo a buscar: 30

Empleado encontrado:
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo Interno
```



- **Captura 4: Mostrar la raíz del árbol**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 3

Raiz del arbol:
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
```

- **Captura 5: Recorrido inorden**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 4

Recorrido Inorden:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo Interno
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```

- **Captura 6: Recorrido preorden**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 5

Recorrido Preorden:
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo Interno
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```



- **Captura 7: Recorrido postorden**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 6

Recorrido Postorden:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo Interno
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
```

- **Captura 8: Mostrar altura del árbol**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 7

Altura del arbol: 3
```

- **Captura 9: Mostrar nodos hoja**

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 8

Nodos hoja:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```



2.8 Conceptos clave

Raíz: Es el primer nodo del árbol. En tu programa, el primer empleado insertado (código 50 - Empresa UTA) se convierte en la raíz. No tiene padre.

Nodo interno: Es un nodo que tiene al menos un hijo (izquierdo o derecho). En tu árbol, los códigos 30 y 70 son nodos internos porque tienen hijos.

Hoja: Es un nodo que no tiene hijos (izquierdo ni derecho). En tu árbol, los códigos 20, 40, 60 y 80 son hojas.

Nivel: Es la distancia desde la raíz hasta un nodo. La raíz está en nivel 0, sus hijos en nivel 1, los nietos en nivel 2, etc.

Altura: Es el número máximo de niveles desde la raíz hasta la hoja más profunda. En tu árbol, la altura es 3 (raíz nivel 0, hasta nivel 2).

2.9 Cómo compilar y ejecutar

1. Descargar o clonar el repositorio

Clonar con Git:

```
git clone https://github.com/kmoyolema3929/G1-ArbolBST-Empresarial
```

Pasos para clonar

1. Navegar a la carpeta donde quieres guardar el proyecto
cd Documentos/Proyectos
2. Clonar el repositorio
git clone https://github.com/kmoyolema3929/G1-ArbolBST-Empresarial.git
3. Entrar a la carpeta del proyecto
cd G1-ArbolBST-Empresarial
4. Verificar que se clono correctamente
ls

Descargar como ZIP:

- Entrar al repositorio en GitHub
- Hacer clic en el boton verde "Code"
- Seleccionar "Download ZIP"
- Extraer el archivo ZIP en una carpeta

2. Ejecutar en Visual Studio Code

Paso 1: Abrir Visual Studio Code

Paso 2: Abrir la carpeta del proyecto

- Ir a "Archivo" -> "Abrir carpeta"
- Seleccionar la carpeta donde descargaste o clonaste el repositorio

Paso 3: Instalar la extension de C++ (si no la tienes)

- Ir a extensiones (Ctrl + Shift + X)
- Buscar "C/C++" de Microsoft
- Hacer clic en "Instalar"



Paso 4: Abrir la terminal integrada

- Presionar Ctrl + Ñ o Ctrl + J
- O ir a "Terminal" -> "Nueva terminal"

Paso 5: Compilar el programa

En la terminal, escribir:

```
g++ src/Nodo.cpp src/ArbolBST.cpp src/main.cpp -o arbol
```

Paso 6: Ejecutar el programa

En Linux / Mac:

```
./arbol
```

En Windows:

```
arbol.exe
```

2.10 Requisitos técnicos

Compilador: g++ (MinGW en Windows, GCC en Linux/Mac)

Tener Git instalado en tu computadora

IDE recomendado: Visual Studio Code

Sistema operativo: Windows, Linux o Mac

Conocimientos previos: Estructura de datos, Árboles BST, C++ básico

2.11 Conclusiones

El Árbol Binario de Búsqueda (BST) permite:

- Organizar jerárquicamente a los empleados
- Búsquedas eficientes en tiempo $O(\log n)$
- Recorridos ordenados automáticos (inorden)
- Identificar fácilmente nodos raíz, internos y hojas

2.12 Recomendaciones

- Agregar mensaje de confirmación al insertar empleado
- Agregar un mensaje cuando el árbol está vacío
- Agregar comentarios en el código

2.13 Referencias bibliográficas

Diapositivas de la asignatura Estructura de Datos - Tema Árboles BST

2.14 Anexos

Enlace repositorio: <https://github.com/kmoyolema3929/G1-ArbolBST-Empresarial>